Module 7

Heaps and Heap Sort

Heap Sort

A binary heap **is a complete binary tree** in which every node satisfies the **heap property** which states that:

if B is a child of A then A<=B

This implies that elements at every node will be either less than or equal to the element at its left and right child.

Thus the root node has the lowest value in the heap.such a heap is commonly known as **min heap.**

Types of heap

- Min heap
- Max heap

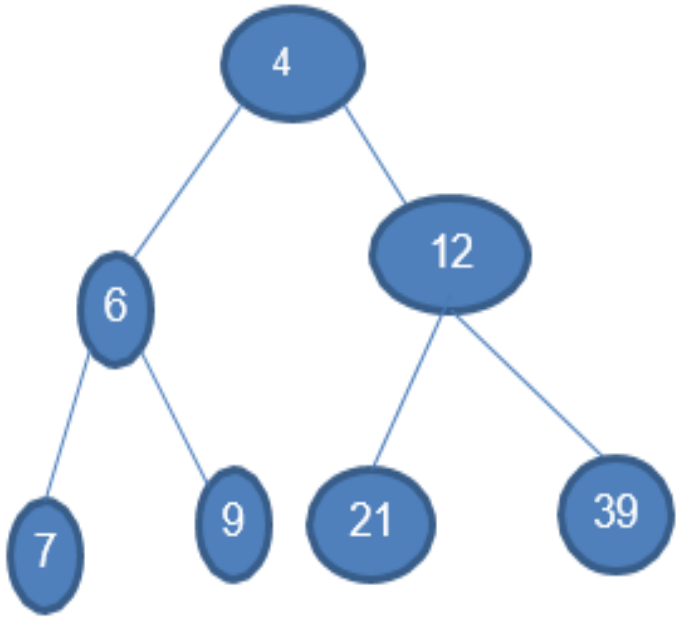
Max heap

A max heap is a tree in which value of each node is greater than or equal to the value of its children node.

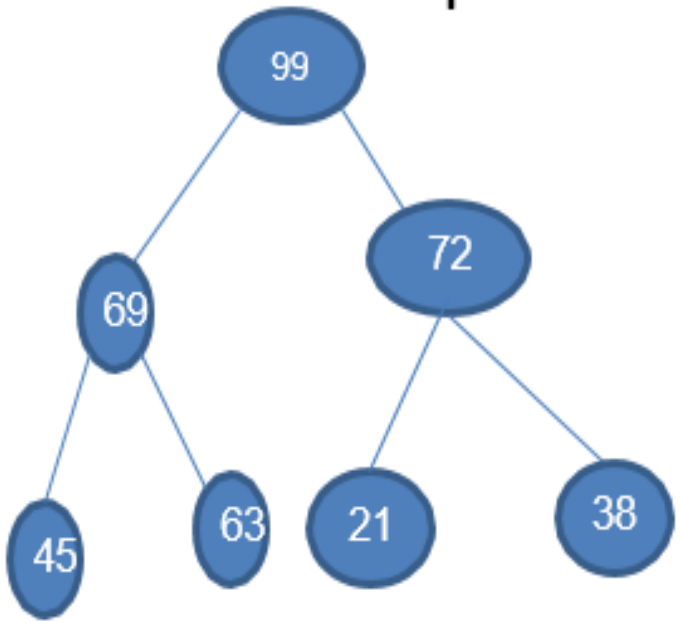
Properties

- **Structure property-** it is a complete binary tree Ie completely filled ,with the exception of the bottom level,which is filled from left to right.
- **Heap order property-** the value stored at a node is greater or equal to the values stored at the children

Min heap



max heap



Heap sort

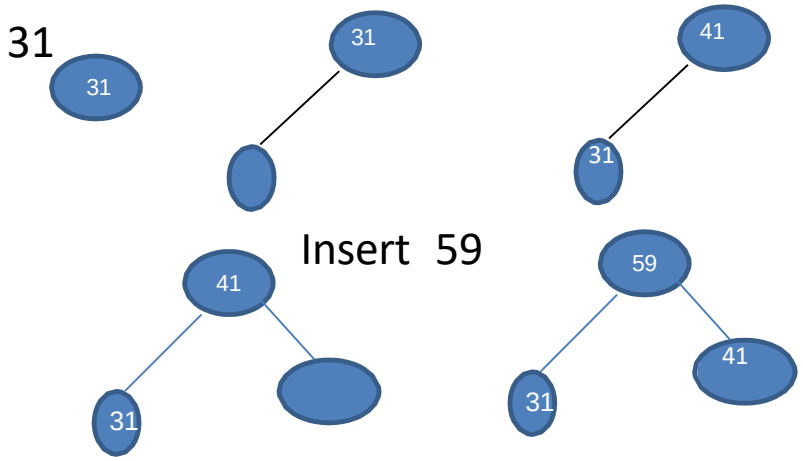
Given an array ARR with n elements, the heap sort algorithm can be used to sort ARR in two phases

In phase 1,build a heap H using the elements of array.

In phase 2,repeatedly delete the root element of the heap formed in phase 1.

Build max heap

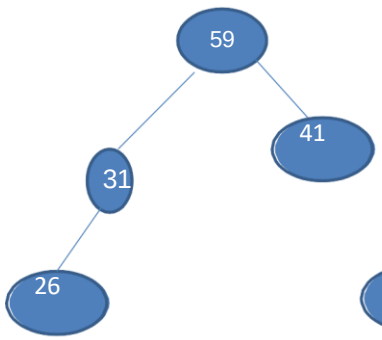
31,41,59,26,53,58,97 insert 41



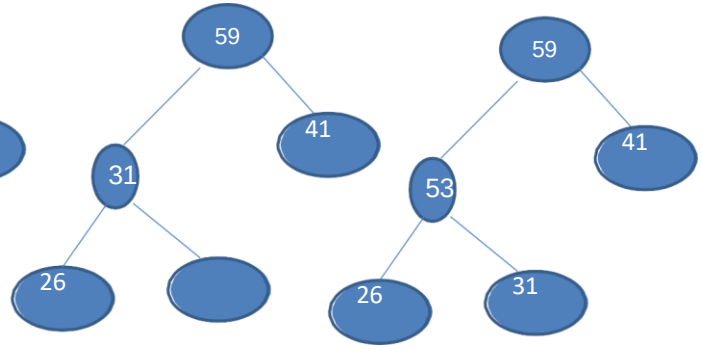
creation

31,41,59,26,53,58,97

Insert 26



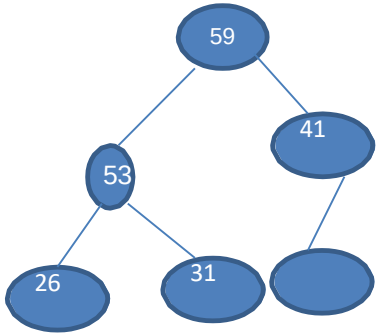
insert 53

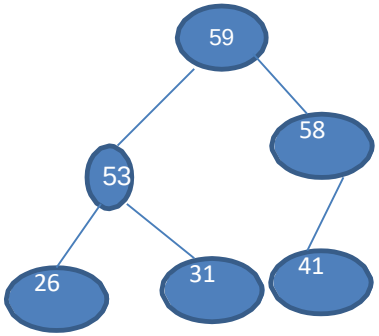


creation

31,41,59,26,53,58,97

Insert 58

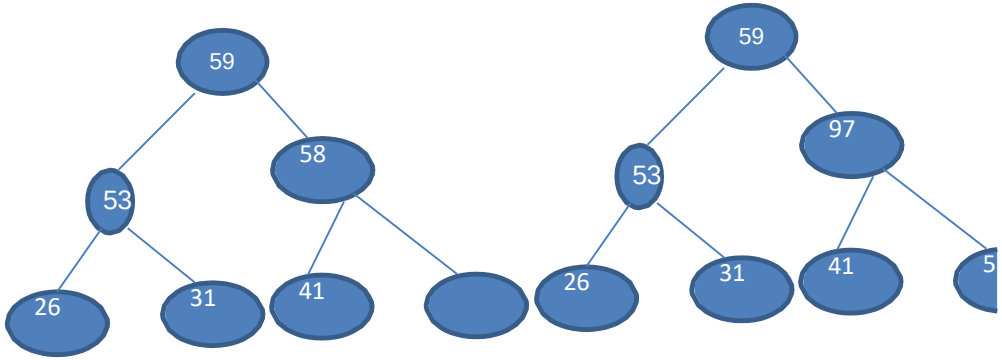




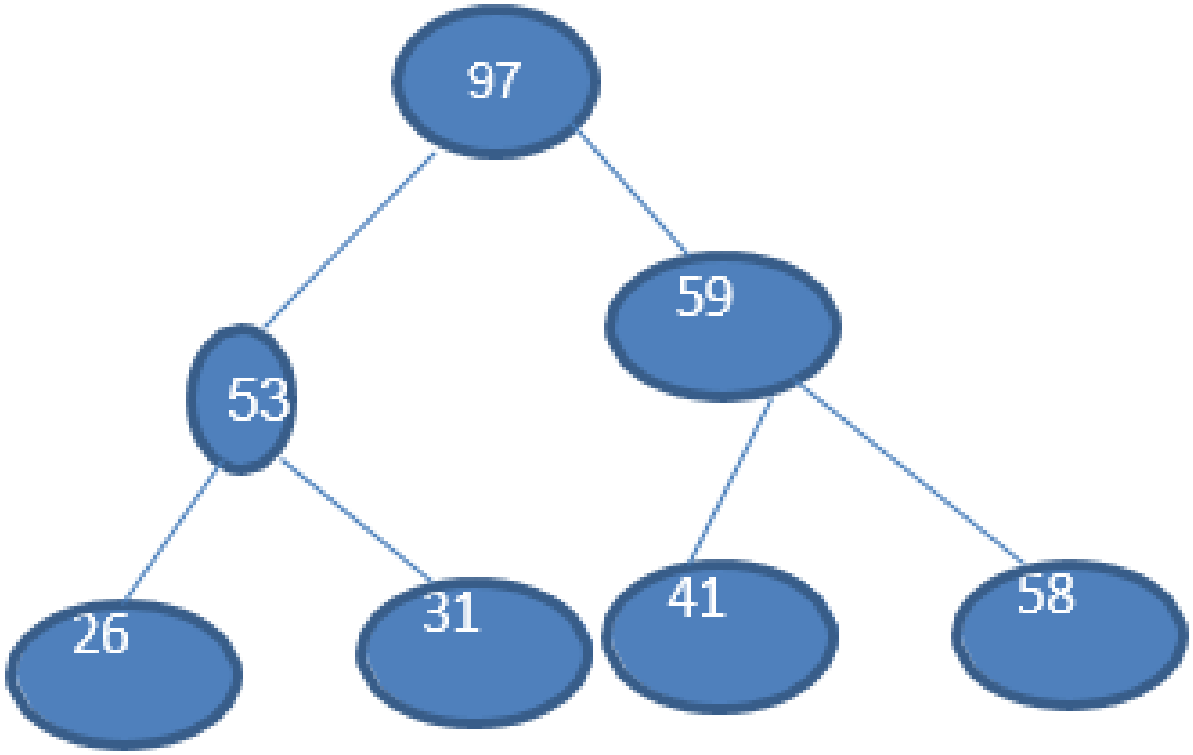
creation

31,41,59,26,53,58,97

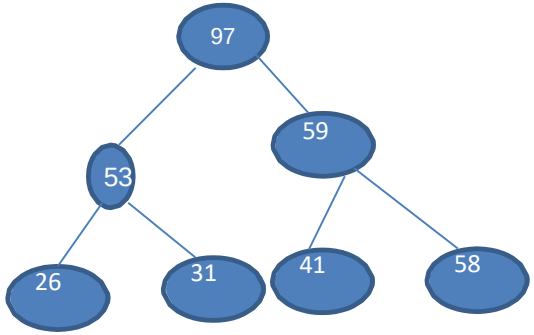
Insert 97



Max Heap is:



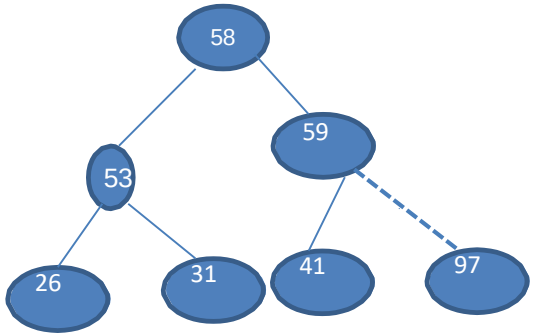
Delete root



97	53	59	26	31	41	58

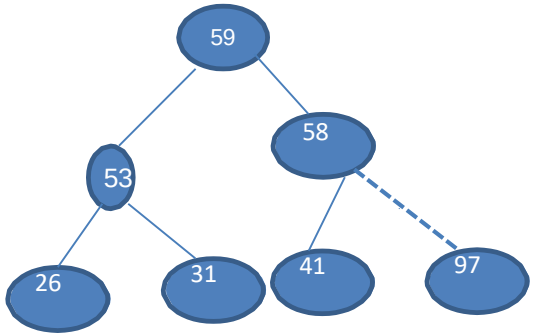
Delete root

Delete 97



Delete root

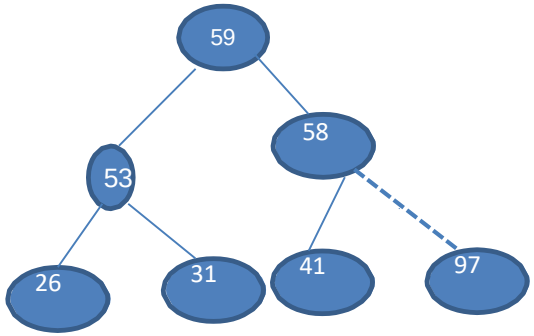
Delete 97



59	53	58	26	31	41	97

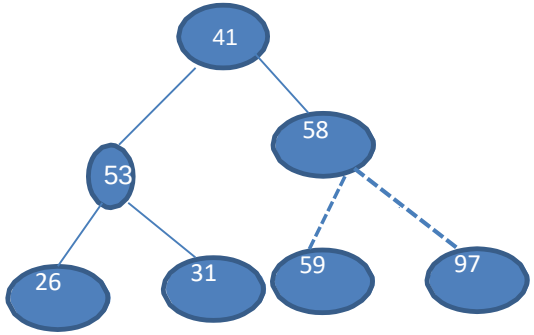
Delete root

Delete 59



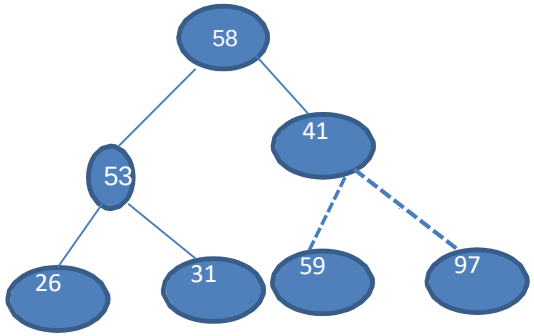
Delete root

Delete 59



Delete root

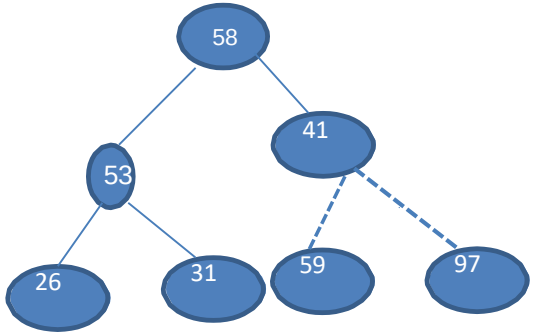
Delete 59



58	53	41	26	31	59	97

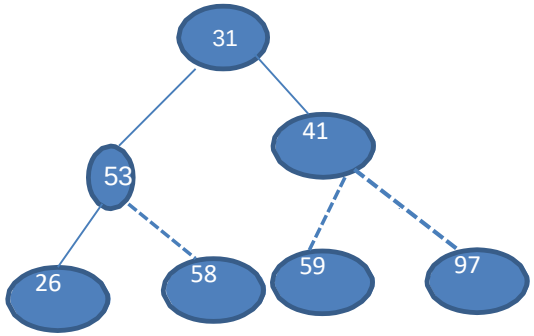
Delete root

Delete 58



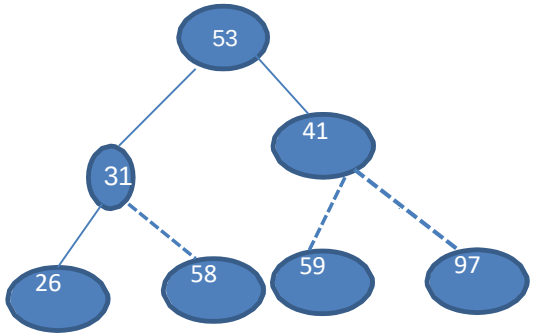
Delete root

Delete 58



Delete root

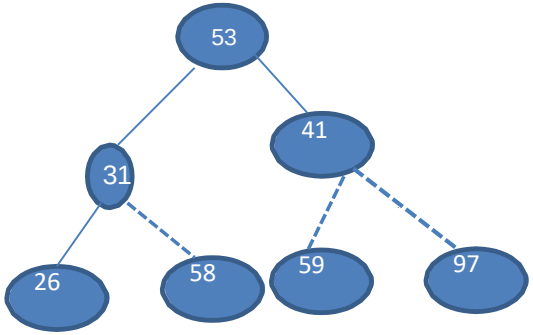
Delete 58



53	31	41	26	58	59	97

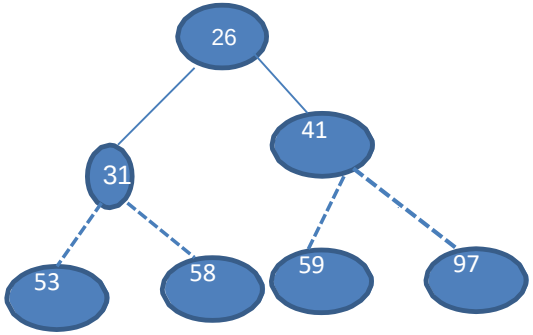
Delete root

Delete 53



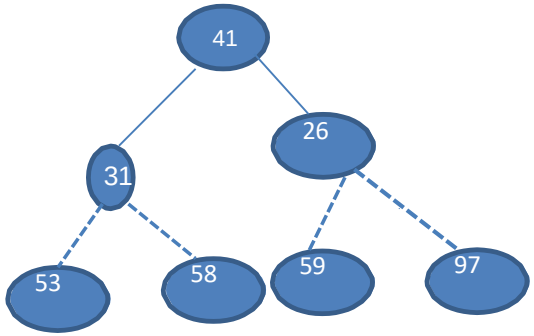
Delete root

Delete 53



Delete root

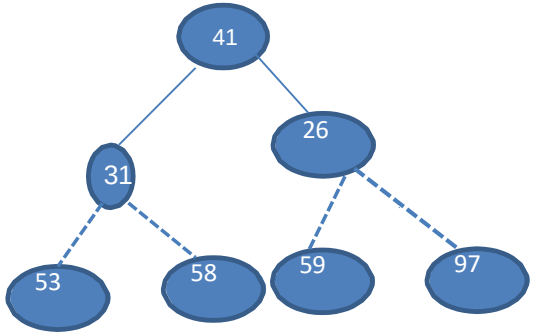
Delete 53



41	31	26	53	58	59	97

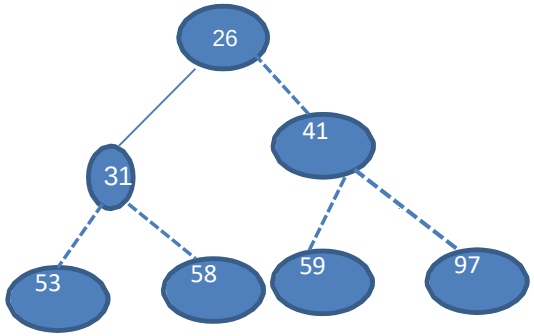
Delete root

Delete 41



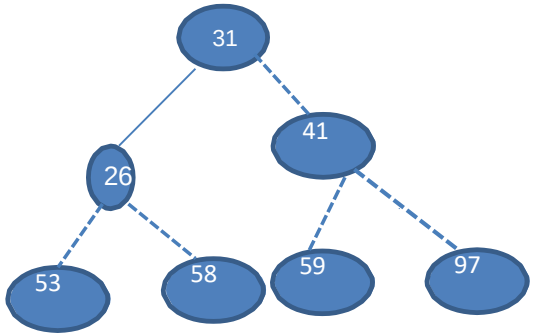
Delete root

Delete 41



Delete root

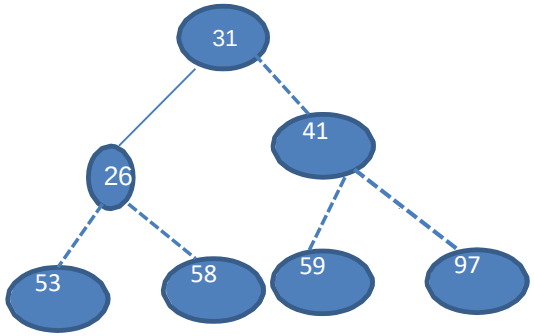
Delete 41



31	26	41	53	58	59	97

Delete root

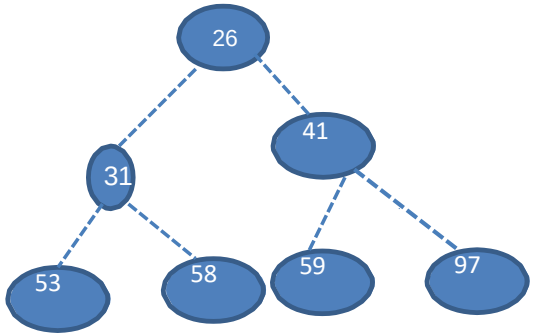
Delete 31



31	26	41	53	58	59	97

Delete root

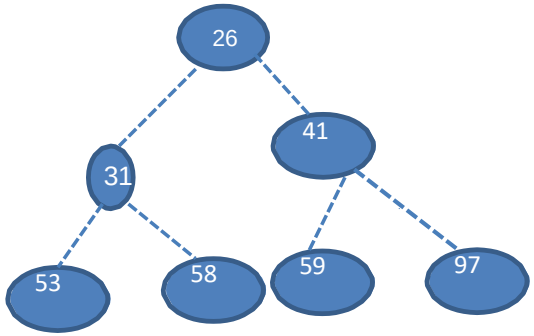
Delete 31



26	31	41	53	58	59	97

Delete root

Delete 26

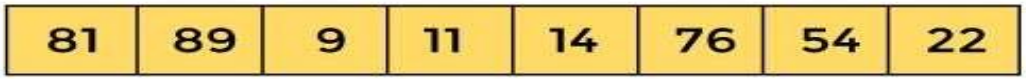


26	31	41	53	58	59	97

Advantage

- It is a simple ,fast and stable sorting algorithm.
- It can be used to sort large sets of data efficiently.

Apply heap sort to sort the following elements



Heap sort - Algorithm

```
void heapify(int arr[], int n, int i)
{
// Find largest among root, left child and right child
            int largest = i;
            int left = 2 * i + 1;
            int right = 2 * i + 2;
            if (left < n && arr[left] > arr[largest])
               largest = left;
// Swap and continue heapifying if root is not largest
            if (right < n && arr[right] > arr[largest])
                largest = right;
             if (largest != i)
              {
                    swap(&arr[i], &arr[largest]);
                    heapify(arr, n, largest);
               }

}
```